

编 号	题 号	标 题	知 识 点	代 码 作 者
例 2-10	UVa 1149	Bin Packing	贪心	陈锋
例 2-11	HDU 2037	今年暑假不 AC	贪心：选择不相交区间	陈锋
例 2-12	UVa 1615	Highway	贪心：区间选点	陈锋
例 2-13	Uva 10382	Watering Grass	贪心：区间覆盖	陈锋
例 2-14	UVa 11729	Commando War	贪心法求最优排列	陈锋

2.3 搜索算法

搜索算法包含多种，其中回溯法的应用范围很广，只要能把待求解的问题分成不太多的步骤，每个步骤又只有不太多的选择，都可以考虑使用回溯法。

例2-15 【下一个排列】排列 (POJ 1833)

给出正整数 n ，则 $1 \sim n$ 这 n 个数可以构成 $n!$ 种排列，把这些排列按照从小到大的顺序（字典顺序）列出，如 $n=3$ 时，列出 $\{1,2,3\}$ ， $\{1,3,2\}$ ， $\{2,1,3\}$ ， $\{2,3,1\}$ ， $\{3,1,2\}$ ， $\{3,2,1\}$ 共6个排列。给出某个排列，求出这个排列的下 k 个排列，如果遇到最后一个排列，则下一个排列为第1个排列，即排列 $\{1,2,3,\dots,n\}$ 。

例如， $n=3$ ， $k=2$ 给出排列 $\{2,3,1\}$ ，则它的下一个排列为 $\{3,1,2\}$ ，下2个排列为 $\{3,2,1\}$ ，因此答案为 $\{3,2,1\}$ 。

【代码实现】

```

// 陈锋
#include <cstdio>
#include <algorithm>
using namespace std;
const int NN = 1024 + 8;
int A[NN];
int main() {
    int T, n, K;
    scanf("%d", &T);
    while (T--) {
        scanf("%d %d", &n, &K);
        for (int i = 0; i < n; ++i) scanf("%d", &A[i]);
        for (int i = 0; i < K; i++) next_permutation(A, A + n);
        for (int i = 0; i < n; ++i) printf("%d ", A[i]);
        puts("");
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》7.2.1节生成1~n的排列
注意：next_permutation会直接改变输入的容器（本题中就是数组A）
同类题目：Ignatius and the Princess II, HDU 1027
*/

```

例2-16 【生成可重集的排列】订单（Orders, CEOI 1999, POJ 1731）

给出不超过200个小写字母，计算这些字母能够组成的所有全排列，并按照字典序输出。

【代码实现】

```

// 陈锋
#include <iostream>
#include <algorithm>
#include <string>
using namespace std;
int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    string s;
    cin >> s;
    sort(s.begin(), s.end());
    cout << s << endl;
    while (next_permutation(s.begin(), s.end()))
        cout << s << endl;
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》7.2.1节生成可重集的排列
注意：对 next_permutation 的调用
*/

```

例 2-17 【BFS 搜索最短路径】Abbott 的复仇（Abbott's Revenge, ACM/ICPC WF 2000, UVa 816）

有一个最多包含 9×9 个交叉点的迷宫。输入起点、离开起点时的朝向和终点，求一条最短路径（多解时任意输出一个即可）。

这个迷宫的特殊之处在于：进入一个交叉点的方向（用N、E、W、S这4个字母分别表示北、东、西、南，即上、右、左、下）不同，允许出去的方向也不同。例如，“1 2 WLF NR ER *”表示交叉点(1, 2)（上数第1行，左数第2列）有3个路标（字符“*”只是结束标志）。如果进入该交叉点时的朝向为W（即朝左），则可以左转（L）或者直行（F）；如果进入时朝向为N或者E，则只能右转（R），如图2-3所示。



图2-3 迷宫及走向

注意：初始状态是“刚刚离开入口”，所以即使出口和入口重合，最短路径也不为空。例如，图2-3中从交叉口(3, 1)进入并到达目标交叉口(3, 3)的一条最短路径为(3, 1), (2, 1), (1, 1), (1, 2), (2, 2), (2, 3), (1, 3), (1, 2), (1, 1), (2, 1), (2, 2), (1, 2), (1, 3), (2, 3), (3, 3)。

【代码实现】

```

// 刘汝佳
#include <cstdio>
#include <cstring>
#include <queue>
#include <vector>
using namespace std;

struct Node {
    int r, c, dir; // 站在(r,c), 面朝方向 dir (0~3 分别表示 N, E, S, W)
    Node(int r = 0, int c = 0, int dir = 0) : r(r), c(c), dir(dir) {}
};

const int NN = 10, dr[] = { -1, 0, 1, 0}, dc[] = {0, 1, 0, -1};
const char *dirs = "NESW", *turns = "FLR"; // 顺时针旋转
int has_edge[NN][NN][4][3], d[NN][NN][4];
int r0, c0, dir, r1, c1, r2, c2;
Node p[NN][NN][4];

int dir_id(char c) { return strchr(dirs, c) - dirs; }
int turn_id(char c) { return strchr(turns, c) - turns; }

Node walk(const Node& u, int turn) {
    int dir = u.dir;
    if (turn == 1) dir = (dir + 3) % 4; // 逆时针
    if (turn == 2) dir = (dir + 1) % 4; // 顺时针
    return Node(u.r + dr[dir], u.c + dc[dir], dir);
}

bool inside(int r, int c) { return r >= 1 && r <= 9 && c >= 1 && c <= 9; }
bool read_case() {
    char s[99], s2[99];
    if (scanf("%s%d%d%s%d%d", s, &r0, &c0, s2, &r2, &c2) != 6) return false;
}

```

```

printf("%s\n", s);
dir = dir_id(s2[0]), r1 = r0 + dr[dir], c1 = c0 + dc[dir];
memset(has_edge, 0, sizeof(has_edge));
for (int r, c; scanf("%d", &r) == 1 && r;) {
    scanf("%d", &c);
    while (scanf("%s", s) == 1 && s[0] != '*') {
        for (size_t i = 1; i < strlen(s); i++)
            has_edge[r][c][dir_id(s[0])][turn_id(s[i])] = 1;
    }
}
return true;
}

void print_ans(Node u) {
    // 从目标结点逆序追溯到初始结点
    vector<Node> nodes;
    while(true) {
        nodes.push_back(u);
        if (d[u.r][u.c][u.dir] == 0) break;
        u = p[u.r][u.c][u.dir];
    }
    nodes.push_back(Node(r0, c0, dir));

    // 打印解, 每行10个
    for (int i = nodes.size() - 1, cnt = 0; i >= 0; i--) {
        if (cnt % 10 == 0) printf(" ");
        printf(" (%d,%d)", nodes[i].r, nodes[i].c);
        if (++cnt % 10 == 0) printf("\n");
    }
    if (nodes.size() % 10 != 0) printf("\n");
}

void solve() {
    queue<Node> q;
    memset(d, -1, sizeof(d));
    Node u(r1, c1, dir);
    d[u.r][u.c][u.dir] = 0;
    q.push(u);
    while (!q.empty()) {
        Node u = q.front();
        q.pop();
        if (u.r == r2 && u.c == c2) {
            print_ans(u);
            return;
        }
        for (int i = 0; i < 3; i++) {
            Node v = walk(u, i);
            if (has_edge[u.r][u.c][u.dir][i] && inside(v.r, v.c) &&
                d[v.r][v.c][v.dir] < 0) {

```

```

        d[v.r][v.c][v.dir] = d[u.r][u.c][u.dir] + 1;
        p[v.r][v.c][v.dir] = u;
        q.push(v);
    }
}
}
puts(" No Solution Possible");
}

int main() {
    while (read_case()) solve();
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》例题 6-14
注意：本题中通过对 4 个方向统一编码，减少了大量的分支判断代码
同类题目：Patrol Robot, ACM/ICPC Hanoi 2006, UVa 1600
*/

```

例2-18 【BFS；路径查找优化】万圣节后的早晨（The Morning after Halloween, Japan 2007, UVa 1601）

$w \times h$ ($w, h \leq 16$) 网格上有 n ($n \leq 3$) 个小写字母（代表鬼）。要求把它们分别移动到对应的大写字母里。每步可以有多个鬼同时移动（均为往上、下、左、右4个方向之一移动），但该步结束之后任何两个鬼不能占用同一个位置，也不能在一步之内交换位置。例如，在如图2-4所示的局面中，一共有4种移动方式（见图2-5）。

```

#####
  ab#
#c##
#####

```

图2-4 题设局面

```

#####  #####  #####  #####
  ab#   a b#   acb#   ab #
#c##   #c##   #  ##   #c##
#####  #####  #####  #####

```

图2-5 4种移动方式

输入保证所有空格连通，所有障碍格也连通，且任何一个 2×2 子网格中至少有一个障碍格。输出最少的步数，且输入保证有解。

【代码实现】

```
// 陈锋
#include <bits/stdc++.h>
using namespace std;
#define _for(i, a, b) for (int i = (a); i < (int)(b); ++i)
typedef long long LL;
const int MAXH = 16 + 4, MAXV = 16 * 16;
const int DX[] = { -1, 1, 0, 0, 0}, DY[] = {0, 0, -1, 1, 0};
int W, H, N, V, ID[16][16], X[MAXV], Y[MAXV];
int D[MAXV][MAXV][MAXV], Src[3], Dest[3];
char MAP[MAXH][MAXH];
vector<int> G[MAXV];
```



```

inline int enc(int a, int b, int c) { return (a << 16) + (b << 8) + c; }

inline bool conflict(int a, int b, int na, int nb) {
    assert(a != b);
    if (na == nb) return true;
    if (na == b && nb == a) return true;
    return false;
}

int bfs() {
    memset(D, -1, sizeof(D));
    queue<int> Q;
    Q.push(enc(Src[0], Src[1], Src[2])); D[Src[0]][Src[1]][Src[2]] = 0;
    while (!Q.empty()) {
        int x = Q.front(), a = x >> 16, b = (x >> 8) & 255, c = x & 255;
        Q.pop();
        int d = D[a][b][c];
        if (a == Dest[0] && b == Dest[1] && c == Dest[2]) return d;
        for (size_t ai = 0; ai < G[a].size(); ai++) {
            int na = G[a][ai];
            for (size_t bi = 0; bi < G[b].size(); bi++) {
                int nb = G[b][bi];
                if (conflict(a, b, na, nb)) continue;
                for (size_t ci = 0; ci < G[c].size(); ci++) {
                    int nc = G[c][ci], &nd = D[na][nb][nc];
                    if (!conflict(a, c, na, nc) && !conflict(b, c, nb, nc) && nd == -1)
                        nd = D[a][b][c] + 1, Q.push(enc(na, nb, nc));
                }
            }
        }
    }
    return -1;
}

int main() {
    while (scanf("%d%d%d", &W, &H, &N) == 3 && W) {
        getchar();
        _for(r, 0, H) fgets(MAP[r], MAXH, stdin);
        V = 0;
        _for(r, 0, H) _for(c, 0, W) {
            char ch = MAP[r][c];
            if (ch == '#') continue;
            ID[r][c] = V, X[V] = r, Y[V] = c;
            if (isupper(ch)) Dest[ch - 'A'] = V;
            if (islower(ch)) Src[ch - 'a'] = V;
            ++V;
        }
        _for(u, 0, V) {
            G[u].clear();

```

```

    int x = X[u], y = Y[u];
    _for(d, 0, 5) {
        int nx = x + DX[d], ny = y + DY[d];
        if (0 <= nx && nx < H && 0 <= ny && ny < W && MAP[nx][ny] != '#')
            G[u].push_back(ID[nx][ny]);
    }
}
if (N <= 2) Src[2] = Dest[2] = V, G[V].clear(), G[V].push_back(V), ++V;
if (N <= 1) Src[1] = Dest[1] = V, G[V].clear(), G[V].push_back(V), ++V;
printf("%d\n", bfs());
}
return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》例题7-9
注意：本题仅仅通过简化图的结构就足够通过时限
*/

```

例2-19 【双向BFS】骑士移动 (Knight Moves, POJ 1915)

编写一个程序，计算一个骑士从棋盘上的一个格子到另一个格子所需的最少步数。骑士一步可以移动到的位置如图2-6所示。其中，黑点表示骑士，灰色格子表示可跳到的位置。

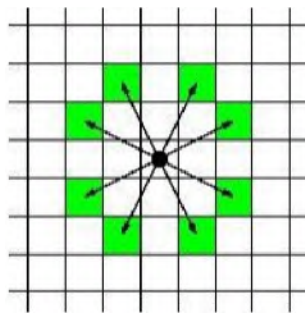


图2-6 骑士可移动的位置

【代码实现】

```

// 陈锋
#include <queue>
#include <cstdio>
#include <cstring>
#include <iostream>
#include <algorithm>
using namespace std;

int DX[] = { 1, 2, 1, 2, -1, -2, -1, -2}, DY[] = { 2, 1, -2, -1, 2, 1, -2, -1}, N;
struct Point {
    int x, y;
    Point(int x = 0, int y = 0) : x(x), y(y) {}
    bool operator==(const Point& p) const {
        return x == p.x && y == p.y;
    }
};
queue<Point> Qs[2];
bool Vis[2][305][305];

bool BFS(int qi) {
    queue<Point> &q = Qs[qi];
    int sz = q.size();
    while (sz--) {
        Point p = q.front();

```

```

    q.pop();
    for (int i = 0; i < 8; ++i) {
        int nx = p.x + DX[i], ny = p.y + DY[i];
        if (nx < 0 || ny < 0 || nx >= N || ny >= N || Vis[qi][nx][ny])
            continue;
        Vis[qi][nx][ny] = true;
        q.push(Point(nx, ny));
        if (Vis[1 ^ qi][nx][ny]) return true;
    }
}
return false;
}

```

```

int solve(const Point& s, const Point& e) {
    if (s == e) return 0;
    memset(Vis, 0, sizeof(Vis));
    while (!Qs[0].empty()) Qs[0].pop();
    while (!Qs[1].empty()) Qs[1].pop();
    Qs[0].push(s), Vis[0][s.x][s.y] = true;
    Qs[1].push(e), Vis[1][e.x][e.y] = true;
    int step = 0;
    while (!Qs[0].empty() || !Qs[1].empty()) {
        bool met = BFS(0);
        step++;
        if (met) return step;
        met = BFS(1), ++step;
        if (met) return step;
    }
    return -1;
}

```

```

int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    Point s, e;
    int t;
    cin >> t;
    while (t--) {
        cin >> N;
        cin >> s.x >> s.y >> e.x >> e.y;
        cout << solve(s, e) << endl;
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》7.5节双向BFS
注意：本题的双向BFS实现对于两个队列的操作，以及异或运算的运用
同类题目：Nightmare II, HDU 3085
*/

```

例 2-20 【八皇后问题】苏丹的继任者 (The Sultan's Successors, UVa 167)

在 8×8 棋盘的每个格子中放置一个数字。计算将8个皇后放到棋盘并且使其不互相攻击的方案中，皇后所在格子上数字之和的最大值。

【代码实现】

```

// 刘汝佳
// 算法：枚举全排列，然后判断是否有两个皇后在同一条对角线上
#include <algorithm>
#include <cstdio>
using namespace std;
const int SZ = 8;
int v[SZ][SZ], P[SZ];

bool judge() {
    bool ok = true;
    for (int i = 0; i < SZ; i++)
        for (int j = 0; j < i; j++)
            if (P[i] - i == P[j] - j || P[i] + i == P[j] + j) return false;
    return true;
}

int main() {
    int T;
    scanf("%d", &T);
    while (T--) {
        for (int i = 0; i < SZ; i++) {
            P[i] = i;
            for (int j = 0; j < SZ; j++) scanf("%d", &v[i][j]);
        }
        int ans = 0;
        do {
            if (!judge()) continue;
            int sum = 0;
            for (int i = 0; i < SZ; i++) sum += v[i][P[i]];
            ans = max(ans, sum);
        } while (next_permutation(P, P + SZ));
        printf("%5d\n", ans);
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》7.4.1节八皇后问题
注意：本题中通过调用 next_permutation 避免了手写递归过程
同类题目：N皇后问题，HDU 2553
*/

```

例2-21 【回溯法和生成测试法的比较】素数环（Prime Ring Problem, UVa 524）

输入正整数 n ，把整数 $1, 2, 3, \dots, n$ 组成一个环，使得相邻两个整数之和均为素数。输出时从整数1开始逆时针排列。同一个环应恰好

输出一次。其中， $n \leq 16$ 。

【样例输入】

6

【样例输出】

```
1 4 3 2 5 6
1 6 5 2 3 4
```

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>
using namespace std;
int used[17], P[] = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31};
void dfs(const size_t N, vector<int>& nums) {
    if (nums.size() == N) {
        for (size_t i = 0; i < N; i++)
            printf("%d%s", nums[i], i == N - 1 ? "\n" : " ");
        return;
    }

    for (size_t i = 1; i <= N; i++)
        if (!used[i] && binary_search(P, P + 11, i + nums.back())) {
            if (nums.size() == N - 1 && !binary_search(P, P + 11, i + nums.front()))
                continue;
            used[i] = 1, nums.push_back(i);
            dfs(N, nums);
            used[i] = 0, nums.pop_back();
        }
}

int main() {
    vector<int> nums({1});
    for (int t = 1, N; scanf("%d", &N) == 1; t++) {
        if (t > 1) puts("");
        printf("Case %d:\n", t);
        fill_n(used, 17, 0), used[1] = 1;
        dfs(N, nums);
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》例题7-4
注意：本题充分使用 vector，简化了递归调用前后的代码
*/

```

例2-22 【回溯法；避免无用判断】困难的串（Krypton Factor, UVa 129)

如果一个字符串包含两个相邻的重复子串，则称它是“容易的串”，其他串称为“困难的串”。例如，BB、ABCDACABCAB、ABCDABCD都是容易的串，而D、DC、ABDAB、CBABCBA都是困难的串。

输入正整数 n 和 L ，输出由前 L 个字符组成的、字典序第 k 小的困难的串。例如，当 $L=3$ 时，前7个困难的串分别为A、AB、ABA、ABAC、ABACA、ABACAB、ABACABA。输入保证答案不超过80个字符。

【样例输入】

```
7 3
30 3
```

【样例输出】

```
ABACABA
ABACABCACBABCABACABCACBACABA
```

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>
using namespace std;
typedef long long LL;
int N, L;
// s[st1...] == s[st2...], size = len
bool rangeEqual(const string& s, int st1, int st2, int len) {
    return equal(s.begin() + st1, s.begin() + st1 + len, s.begin() + st2);
}

bool dfs(int& k, string& cur, string& ans) {
    if (!cur.empty() && ++k == N) {
        ans = cur;
        return true;
    }
    for (char ch = 'A'; ch < 'A' + L; ch++)
        if (cur.empty() || cur.back() != ch) {
            cur += ch;
            bool valid = true;
            int sz = cur.size();
            string::iterator sit = cur.begin();
            for (int l = 2; l <= sz / 2; l++) {
                // 尝试长度为 j*2 的后缀
                if (rangeEqual(cur, sz - l, sz - l * 2, l)) {
                    valid = false;
                    // 后一半等于前一半, 方案不合法
                    break;
                }
            }
            // 递归搜索, 如果已经找到解, 则直接退出
            if (valid && dfs(k, cur, ans)) return true;
            cur.pop_back();
        }
    return false;
}

int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    while (cin >> N >> L && N && L) {
        string cur, ans;
        int k = 0;
        dfs(k, cur, ans);
        for (size_t i = 0; i < ans.size(); i++) {
            // 输出答案
            cout << ans[i];
            if (i == 63 || i == ans.size() - 1) cout << endl;
            else if (i % 4 == 3) cout << " ";
        }
        cout << ans.size() << endl;
    }
    return 0;
}

/*
算法分析请参考:《算法竞赛入门经典(第2版)》例题7-5
注意:对字符串子串进行比较可以使用 STL 中的 equal 函数,避免手写循环
同类题目: Bandwidth, UVa 140
*/

```

例2-23 【迭代加深搜索】埃及分数问题 (Egyptian Fractions, BIO 1997 Round 1 Question 3)

在古埃及，人们使用单位分数的和（即 $1/a$ 的和， a 是自然数）表示一切有理数。例如， $2/3=1/2+1/6$ ，但不允许 $2/3=1/3+1/3$ ，因为在加数中不允许有相同的分母。

对于一个分数 a/b ，表示方法有很多种，其中加数少的比加数多得好，如果加数个数相同，则最小的分数越大越好。例如， $19/45=1/5+1/6+1/18$ 是最优方案。

输入整数 a, b ($0 < a < b < 500$)，试编程计算最佳表达式。

【代码实现】

```

// 陈锋
// http://www.olympiad.org.uk/papers/1997/bio/bio97rlq3.html
#include <bits/stdc++.h>
using namespace std;
typedef long long LL;
int A, B;
vector<LL> D, Ans;
bool better() { // D 是否比 Ans 好
    if (Ans.empty()) return true;
    assert(D.size() == Ans.size());
    for (int i = D.size() - 1; i >= 0; i--)
        if (D[i] != Ans[i]) return D[i] < Ans[i];
    return false;
}
//  $a/b + 1/c = A/B \rightarrow 1/c \leq A/B - a/b = (A*b - a*B) / (B*b) \rightarrow c \geq (B*b) / (A*b - a*B)$ 
inline LL firstc(LL a, LL b, LL last) {
    return max(B * b / (A * b - a * B), last + 1);
}
void dfs(LL a, LL b, const int d, const int maxd) {
    if (d > maxd || a * B > A * b) return; //  $a/b > A/B$ 
    if (A * b == a * B) { //  $A/B == a/b$ 
        if (better()) Ans = D;
        return;
    }
    LL lastc = D.empty() ? 1LL : D.back(); //  $a/b + (maxd - d)/c < A/B$ : break
    for (LL c = firstc(a, b, lastc); a * B * c + (maxd - d) * B * b >= A * b * c; ++c)
        D.push_back(c), dfs(a * c + b, b * c, d + 1, maxd), D.pop_back();
}

int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    cin >> A >> B;
    for (int maxd = 2; maxd <= 100; maxd++) {
        dfs(0, 1, 0, maxd);
        if (!Ans.empty()) {
            for (size_t i = 0; i < Ans.size(); i++)
                printf("%s%lld", (i ? " " : ""), Ans[i]);
            puts("");
            break;
        }
    }
    return 0;
}

```

例2-24 【IDA*；迭代加深搜索】埃及分数（Egyptian Fractions (HARD version), Rujia Liu' s Present 6, UVa 12558)

把 a/b 写成不同的埃及分数之和，要求项数尽量少，但最小的分数尽量大，然后第2小的分数尽量大……另外，有 k ($0 \leq k \leq 5$) 个数不能用作分母。例如， $k=0$ 时 $5/121=1/33+1/121+1/363$ ，不能使用。 $k=33$ 时，最优解为 $5/121=1/45+1/55+1/1089$ 。

输入保证 $2 \leq a < b \leq 876$ ， $\gcd(a,b)=1$ ，且会挑选比较容易求解的数据。

【代码实现】

```
// 陈锋
#include <bits/stdc++.h>
using namespace std;
typedef long long LL;
int A, B, K;
set<int> R;
LL gcd(LL a, LL b) { return b == 0 ? a : gcd(b, a % b); }

bool better(const vector<LL>& D, const vector<LL>& Ans) {
    if (Ans.empty()) return true;
    size_t sz = D.size();
    assert(sz == Ans.size());
    for (int i = sz - 1; i >= 0; i--)
```

```

    if (D[i] != Ans[i]) return D[i] < Ans[i];
    return false;
}

inline LL get_first(LL a, LL b, LL last) {
    //  $a/b + 1/c = A/B$ 
    //  $1/c \leq A/B - a/b = (A*b - a*B) / (B*b)$ 
    //  $c \geq (B*b) / (A*b - a*B)$ 
    return max((B * b) / (A * b - a * B), last + 1);
}

void dfs(LL a, LL b, const int d, const int maxd, vector<LL>& D,
        vector<LL>& Ans) {
    if (d > maxd) return;
    if (a * B > A * b) return;
    if (a == A && b == B) {
        if (better(D, Ans)) Ans = D;
        return;
    }
    LL deno = get_first(a, b, D.empty() ? 1LL : D.back());
    while (true) {
        if (a * B * deno + (maxd - d) * B * b < A * b * deno)
            break; //  $a/b + (maxd-d)/deno < A/B$ 
        if (!R.count(deno)) {
            LL na = a * deno + b, nb = b * deno, g = gcd(na, nb);
            D.push_back(deno);
            dfs(na / g, nb / g, d + 1, maxd, D, Ans);
            D.pop_back();
        }
        deno++;
    }
    return;
}

int main() {
    int T; scanf("%d", &T);
    for (int t = 1; t <= T; t++) {
        R.clear(), scanf("%d%d%d", &A, &B, &K);
        for (int i = 0, x; i < K; i++) scanf("%d", &x), R.insert(x);
        for (int maxd = 2; maxd <= 100; maxd++) {
            vector<LL> D, Ans;
            dfs(0, 1, 0, maxd, D, Ans);
            if (Ans.empty()) continue;
            printf("Case %d: %d/%d=", t, A, B);
            for (size_t i = 0; i < Ans.size(); i++)
                printf("1/%lld%c", Ans[i], (i == Ans.size() - 1) ? '\n' : '+');
            break;
        }
    }
}

```

```

    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——习题与解答》习题 7-7
注意：本题将容易出错的两个子逻辑单独封装，以方便代码检查
同类题目：Editing a Book, UVa 11212
          Power Calculus, UVa 1374
          DNA sequence, HDU 1560
*/

```

例 2-25 【A*; K 短路问题】 Remmarguts 的约会 (Remmarguts' Date, POJ 2449)

给出一个包含 N 个结点、 M 条边的带权有向图，结点编号为 $1 \sim N$ 。给出两个点 S 和 T ，计算所有从 S 到 T 的路径中，总长度为第 K 短的路径的长度。

【代码实现】

```

// 陈锋
#include <iostream>
#include <algorithm>
#include <vector>
#include <cstring>
#include <queue>
const int NN = 1000 + 4, INF = 0x3f3f3f3f;
using namespace std;

typedef pair<int, int> P;
struct Edge {
    int v, w;
    Edge(int t, int l): v(t), w(l) {}
};

int N, TD[NN];
vector<Edge> G[NN], Rev[NN];
typedef Edge HeapNode;

struct cmpT { // 计算到 T 的最短路径
    bool operator()(const HeapNode &a, const HeapNode &b) {
        return a.w > b.w;
    }
};

void dijkstra(int s) { // 求 s 到任一点的最短路径，使用反向边
    priority_queue<HeapNode, vector<HeapNode>, cmpT> q;
    fill_n(TD, N + 1, INF), TD[s] = 0, q.push(HeapNode(s, 0));
    while (!q.empty()) {
        int u = q.top().v, w = q.top().w;
        q.pop();
        if (TD[u] < w) continue;
        for (size_t i = 0; i < Rev[u].size(); i++) {
            const Edge& e = Rev[u][i];
            if (w + e.w < TD[e.v])

```



```

        TD[e.v] = w + e.w, q.push(HeapNode(e.v, TD[e.v]));
    }
}

struct cmpAstar { // A*使用
    bool operator()(const HeapNode &a, const HeapNode &b) {
        return a.w + TD[a.v] > b.w + TD[b.v];
    }
};

int aStar(int S, int T, int K) {
    if (TD[S] == INF) return -1; // 不连通
    int cnt = 0;
    if (S == T) --cnt; // 出发时不作数，核减1次
    priority_queue<HeapNode, vector<HeapNode>, cmpAstar> q;
    q.push(HeapNode(S, 0));
    while (!q.empty()) {
        int u = q.top().v, d = q.top().w;
        q.pop();
        if (u == T && ++cnt == K) return d; // 第K短路找到了
        for (size_t i = 0; i < G[u].size(); i++) {
            const Edge& e = G[u][i];
            q.push(Edge(e.v, d + e.w));
        }
    }
    return -1;
}

int main() {
    std::ios::sync_with_stdio(false);
    int M, S, T, K;
    cin >> N >> M;
    for (int i = 0, a, b, w; i < M; i++) {
        cin >> a >> b >> w;
        G[a].push_back(Edge(b, w)), Rev[b].push_back(Edge(a, w));
    }
    cin >> S >> T >> K;
    dijkstra(T);
    cout << aStar(S, T, K) << endl;
    return 0;
}

/*
注意：领会本题中对于最短路径的使用
同类题目：path, HDU 6705
*/

```

例2-26 【棋盘皇后覆盖】守卫棋盘 (Guarding the Chessboard, UVa 11214)

输入一个 $n \times m$ 棋盘 ($n, m < 10$)，某些格子有标记。用最少的皇后守卫（即占据或者攻击）所有带标记的格子。

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>
#define _for(i, a, b) for (int i = (a); i < (int)(b); ++i)
using namespace std;

int n, m;
struct Point {
    int x, y;
    Point(int x = 0, int y = 0) : x(x), y(y) {}
};
typedef Point Vector;
Vector operator+(const Vector& A, const Vector& B) {
    return Vector(A.x + B.x, A.y + B.y);
}

const int MAXM = 9;
Vector dirs[] = {{-1, -1}, {1, 1}, {1, -1}, {-1, 1}, {1, 0}, {-1, 0}, {0, 1}, {0, -1}};
// 8个方向向量
bool isValid(const Point& p) {
    return p.x >= 0 && p.x < n && p.y >= 0 && p.y < m;
}

struct Grid {
    int bits[4];
    inline void clear() { memset(bits, 0, sizeof(bits)); }
    inline void set(int r, int c) {
        int l = r * m + c;
        bits[l / 32] |= (1 << (l & 31));
    }

    inline bool canCover(const Grid& g) const {
        return (bits[0] & g.bits[0]) == g.bits[0] &&
            (bits[1] & g.bits[1]) == g.bits[1] &&
            (bits[2] & g.bits[2]) == g.bits[2];
    }

    Grid() { clear(); }
};

// [i,j]如果有皇后, 所有被其覆盖的布局就是 covers[i * m + j]
Grid covers[MAXM * MAXM + 3], target;

void dfs(int ci, const Grid& g, int depth, int& best) {
    if (depth >= best) return;
    if (g.canCover(target)) {
        best = min(best, depth);
    }
}

```

```

        return;
    }
    if (ci > n * m || depth + 1 > best) return;

    dfs(ci + 1, g, depth, best);
    Grid ng = g;
    int* cb = covers[ci].bits;
    ng.bits[0] |= cb[0], ng.bits[1] |= cb[1], ng.bits[2] |= cb[2];
    dfs(ci + 1, ng, depth + 1, best);
}

int main() {
    string line;
    for (int k = 1; cin >> n >> m && n && m; k++) {
        target.clear();
        memset(covers, 0, sizeof(covers));
        _for(i, 0, n) {
            cin >> line;
            assert(line.size() == m);
            _for(j, 0, m) {
                if (line[j] == 'X') target.set(i, j);
                _for(di, 0, 8) for (Point pc(i, j); isValid(pc); pc = pc + dirs[di])
                    covers[i * m + j].set(pc.x, pc.y);
            }
        }

        int best = 6;
        Grid g;
        dfs(0, g, 0, best);
        cout << "Case " << k << ": " << best << endl;
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——习题与解答》习题 7-10
注意：本题中如何使用一个长度为 4 的一维数组来对整个棋盘状态进行压缩存储
*/

```

例2-27 【极大极小过程；Alpha-beta剪枝；对抗搜索】纸牌房屋 (House of Cards, WF 2009, UVa 1085)

有一副牌，共有红、黑两种花色，每种花色有 M 张牌。把这 $2M$ 张牌按照某种顺序排好，取出前8张牌拼成如图2-7所示的“房子”状（有4个峰和3个谷），剩下的牌正面朝上排列好放在桌面上，这样两个游戏者都能知道接下来的每轮各是什么牌。

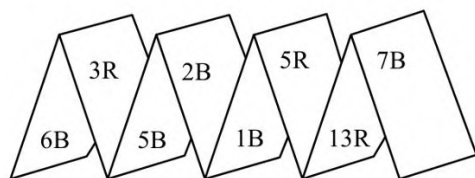


图2-7 纸牌房屋

Axel和Birgit两人进行游戏，其中Axel拥有红色牌，Birgit拥有黑色牌。拥有牌的颜色与图2-7中左数第一张牌的颜色相同的游戏者为先手。Birgit为先手，因为第一张牌6B是黑色的。

游戏者轮流操作。每次操作都取出剩下的左数第一张牌，然后进行以下3种操作之一。

- ☐ 拿在手里（成为“拿着的牌”）。
- ☐ 在两个相邻峰之间搭一张牌（使用拿着的牌或者新取的牌），作为一层楼面。剩下的一张（如果有的话）拿在手里。
- ☐ 将两张牌搭在一张楼面牌上，形成一个新的峰（其中一张牌一定是拿着的牌）。

注意，不是所有情况下这3种操作都是可行的。游戏规定手里最多只能拿一张牌，因此第一种操作可行的前提是手里没有牌。

每当组成一个新的三角形（加楼面牌时会形成一个向下的三角形，加峰时会形成一个向上的三角形）时，计分如下：首先数一数组成三角形的3张牌中哪种花色较多，然后给该游戏者加分，分值为这3张牌的点数之和。

在上面的例子中，如果Birgit在中间的谷上放一张（11R）作为楼面牌，其将得到14分；如果其把这张牌放在左边的谷上，Axel将得到19分；如果放到右边的谷上，Axel将得到29分。

当桌面上没有正面朝上的游戏牌时，游戏结束。如果一个游戏者手里拿着牌，那么还将对他的得分进行最后的调整：如果此牌的花色和自己拥有的牌的颜色相同，加分；否则减分。加减的分值均为该牌的点数。

假定两位游戏者都足够聪明，你的任务是计算某个游戏者最多可以比对手高多少分。

【代码实现】

```
// 刘汝佳
#include <cstdio>
#include <cstring>
#include <algorithm>
#include <vector>
using namespace std;

const int UP = 0, FLOOR = 1, DOWN = 2, maxn = 20;
int n, deck[maxn*2];
struct State {
    int card[8], type[8];          // 两张相同的 FLOOR 牌代表一张真实的 FLOOR 牌
    int hold[2], pos, score;       // MAX 游戏者（即 Axel）的得分
    State child() const {
        State s;
        memcpy(&s, this, sizeof(s));
        s.pos = pos + 1;
        return s;
    }
    State() {
        for(int i = 0; i < 8; i++) {
            card[i] = deck[i];
            type[i] = i % 2 == 0 ? UP : DOWN;
        }
    }
};
```

```

    hold[0] = hold[1] = score = 0;
    pos = 8;
}

bool isFinal() {
    if(pos == 2*n) {
        score += hold[0] + hold[1];
        hold[0] = hold[1] = 0;
        return true;
    }
    return false;
}

int getScore(int c1, int c2, int c3) const {
    int S = abs(c1) + abs(c2) + abs(c3);
    int cnt = 0;
    if(c1 > 0) cnt++; if(c2 > 0) cnt++; if(c3 > 0) cnt++;
    return cnt >= 2 ? S : -S;
}

void expand(int player, vector<State>& ret) const {
    int cur = deck[pos];

    // 决策1: 拿在手里
    if(hold[player] == 0) {
        State s = child();
        s.hold[player] = cur;
        ret.push_back(s);
    }

    // 决策2: 摆楼面牌
    for(int i = 0; i < 7; i++) if(type[i] == DOWN && type[i+1] == UP) {
        // 用当前的牌
        State s = child();
        s.score += getScore(card[i], card[i+1], cur);
        s.type[i] = s.type[i+1] = FLOOR;
        s.card[i] = s.card[i+1] = cur;
        ret.push_back(s);

        if(hold[player] != 0) {
            // 用手里的牌
            State s = child();
            s.score += getScore(card[i], card[i+1], hold[player]);
            s.type[i] = s.type[i+1] = FLOOR;
            s.card[i] = s.card[i+1] = hold[player];
            s.hold[player] = cur;
            ret.push_back(s);
        }
    }
}

```

```

// 决策 3: 新的山峰
if(hold[player] != 0)
    for(int i = 0; i < 7; i++) if(type[i] == FLOOR && type[i+1] == FLOOR && card[i]
== card[i+1]) {
        State s = child();
        s.score += getScore(card[i], hold[player], cur);
        s.type[i] = UP; s.type[i+1] = DOWN;
        s.card[i] = cur; s.card[i+1] = hold[player]; s.hold[player] = 0;
        ret.push_back(s);

        swap(s.card[i], s.card[i+1]);
        ret.push_back(s);
    }
}
};

// 带 alpha-beta 剪枝的对抗搜索
int alphabeta(State& s, int player, int alpha, int beta) {
    if(s.isFinal()) return s.score; // 终态

    vector<State> children;
    s.expand(player, children); // 扩展子结点

    int n = children.size();
    for(int i = 0; i < n; i++) {
        int v = alphabeta(children[i], player^1, alpha, beta);
        if(!player) alpha = max(alpha, v); else beta = min(beta, v);
        if(beta <= alpha) break; // alpha-beta 剪枝
    }
    return !player ? alpha : beta;
}

const int INF = 1e9;

int main() {
    int kase = 0;
    char P[10];
    while(scanf("%s", P) == 1 && P[0] != 'E') {
        scanf("%d", &n);
        for(int i = 0; i < n*2; i++) {
            char ch;
            scanf("%d%c", &deck[i], &ch);
            if(ch == 'B') deck[i] = -deck[i];
        }
        State initial;
        int first_player = deck[0] > 0 ? 0 : 1, score = alphabeta(initial, first_player,
-1e9, 1e9);
        if(P[0] == 'B') score = -score;
        printf("Case %d: ", ++kase);
    }
}

```



```

    if(score == 0) printf("Axel and Birgit tie\n");
    else if(score > 0) printf("%s wins %d\n", P, score);
    else printf("%s loses %d\n", P, -score);
}
return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 6.3 节例题 10
*/

```

例2-28 【数独； Dancing Links】数独（Sudoku, SEERC 2006, POJ 3076)

求解16行、16列数独问题的一组解。给定一个16×16的字母方阵，要求在这样的空格子填上A~P中的一个字符，并满足以下条件。

- (1) 在每行中A~P恰好各出现一次。
- (2) 在每列中A~P恰好各出现一次。
- (3) 粗线分隔出的每个4×4子方阵（一共有4×4个）中，A~P恰好各出现一次。

该数独问题及其解如图2-8所示。

		A				C					O		I
	J			A	B		P		C	G	F		H
		D			F	I		E					P
	G		E	L		H				M		J	
				E				C			G		
	I			K		G	A		B			E	J
	D		G	P			J		F			A	
		E			C	B			D	P			O
	E			F		M			D		L	K	A
		C							O		I		L
	H		P		C			F		A		B	
			G			O	D			J			H
	K				J				H		A	P	L
			B			P			E		K		A
		H			B			K		F	I		C
			F				C		D			H	N

F	P	A	H	M	J	E	C	N	L	B	D	K	O	G	I
O	J	M	I	A	N	B	D	P	K	C	G	F	L	H	E
L	N	D	K	G	F	O	I	J	E	A	H	M	B	P	C
B	G	C	E	L	K	H	P	O	F	I	M	A	J	D	N
M	F	H	B	E	L	P	O	A	C	K	J	G	N	I	D
C	I	L	N	K	D	G	A	H	B	M	O	P	E	F	J
D	O	G	P	I	H	J	M	F	N	L	E	C	A	K	B
J	E	K	A	F	C	N	B	G	I	D	P	L	H	O	M
E	B	O	F	P	M	I	J	D	G	H	L	N	K	C	A
N	C	J	D	H	B	A	E	K	M	O	F	I	G	L	P
H	M	P	L	C	G	K	F	I	A	E	N	B	D	J	O
A	K	I	G	N	O	D	L	B	P	J	C	E	F	M	H
K	D	E	M	J	I	F	N	C	H	G	A	O	P	B	L
G	L	B	C	D	P	M	H	E	O	N	K	J	I	A	F
P	H	N	O	B	A	L	K	M	J	F	I	D	C	E	G
I	A	F	J	O	E	C	G	L	D	P	B	H	M	N	K

图2-8 数独求解

【代码实现】

```
// 刘汝佳
#include <cstdio>
#include <cstring>
#include <vector>

using namespace std;

const int maxr = 5000;
const int maxn = 2000;
const int maxnode = 20000;

// 行编号从 1 开始，列编号为 1~n，结点 0 是表头结点，结点 1~n 是各列顶部的虚拟结点
struct DLX {
    int n, sz; // 列数，结点总数
```

```

int S[maxn]; // 各列结点数

int row[maxnode], col[maxnode]; // 各结点行列编号
int L[maxnode], R[maxnode], U[maxnode], D[maxnode]; // 十字链表

int ansd, ans[maxr]; // 解

void init(int n) { // n是列数
    this->n = n;

    // 虚拟结点
    for(int i = 0 ; i <= n; i++) {
        U[i] = i; D[i] = i; L[i] = i-1, R[i] = i+1;
    }
    R[n] = 0; L[0] = n;

    sz = n + 1;
    memset(S, 0, sizeof(S));
}

void addRow(int r, vector<int> columns) {
    int first = sz;
    for(int i = 0; i < columns.size(); i++) {
        int c = columns[i];
        L[sz] = sz - 1; R[sz] = sz + 1; D[sz] = c; U[sz] = U[c];
        D[U[c]] = sz; U[c] = sz;
        row[sz] = r; col[sz] = c;
        S[c]++; sz++;
    }
    R[sz - 1] = first; L[first] = sz - 1;
}

// 顺着链表A, 遍历除s外的其他元素
#define FOR(i,A,s) for(int i = A[s]; i != s; i = A[i])

void remove(int c) {
    L[R[c]] = L[c];
    R[L[c]] = R[c];
    FOR(i,D,c)
        FOR(j,R,i) { U[D[j]] = U[j]; D[U[j]] = D[j]; --S[col[j]]; }
}

void restore(int c) {
    FOR(i,U,c)
        FOR(j,L,i) { ++S[col[j]]; U[D[j]] = j; D[U[j]] = j; }
    L[R[c]] = c;
    R[L[c]] = c;
}

```

```

// d 为递归深度
bool dfs(int d) {
    if (R[0] == 0) { // 找到解
        ansd = d; // 记录解的长度
        return true;
    }

    // 找 S 最小的列 c
    int c = R[0]; // 第一个未删除的列
    FOR(i,R,0) if(S[i] < S[c]) c = i;

    remove(c); // 删除第 c 列
    FOR(i,D,c) { // 用结点 i 所在行覆盖第 c 列
        ans[d] = row[i];
        FOR(j,R,i) remove(col[j]); // 删除结点 i 所在行能覆盖的所有其他列
        if(dfs(d+1)) return true;
        FOR(j,L,i) restore(col[j]); // 恢复结点 i 所在行能覆盖的所有其他列
    }
    restore(c); // 恢复第 c 列

    return false;
}

bool solve(vector<int>& v) {
    v.clear();
    if(!dfs(0)) return false;
    for(int i = 0; i < ansd; i++) v.push_back(ans[i]);
    return true;
}

};

////////// 题目相关
#include<cassert>

DLX solver;

const int SLOT = 0;
const int ROW = 1;
const int COL = 2;
const int SUB = 3;

// 行/列的统一编解码函数。从 1 开始编号
int encode(int a, int b, int c) {
    return a*256+b*16+c+1;
}

void decode(int code, int& a, int& b, int& c) {
    code--;

```

```

    c = code%16; code /= 16;
    b = code%16; code /= 16;
    a = code;
}

char puzzle[16][20];

bool read() {
    for(int i = 0; i < 16; i++)
        if(scanf("%s", puzzle[i]) != 1) return false;
    return true;
}

int main() {
    int kase = 0;
    while(read()) {
        if(++kase != 1) printf("\n");
        solver.init(1024);
        for(int r = 0; r < 16; r++)
            for(int c = 0; c < 16; c++)
                for(int v = 0; v < 16; v++)
                    if(puzzle[r][c] == '-' || puzzle[r][c] == 'A'+v) {
                        vector<int> columns;
                        columns.push_back(encode(SLOT, r, c));
                        columns.push_back(encode(ROW, r, v));
                        columns.push_back(encode(COL, c, v));
                        columns.push_back(encode(SUB, (r/4)*4+c/4, v));
                        solver.addRow(encode(r, c, v), columns);
                    }

        vector<int> ans;
        assert(solver.solve(ans));

        for(int i = 0; i < ans.size(); i++) {
            int r, c, v;
            decode(ans[i], r, c, v);
            puzzle[r][c] = 'A'+v;
        }
        for(int i = 0; i < 16; i++)
            printf("%s\n", puzzle[i]);
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 6.3.3 节例题 11
*/

```

例2-29 【数位统计；DFS】数字游戏（牛客NC 50517）

科协里最近很流行数字游戏。某人命名了一种不降数，这种数字必须满足从左到右各位数字呈小于等于的关系，如123，446。现在大家决定玩一个游戏，指定一个整数闭区间 $[a,b]$ ，问这个区间内有多少个不降数。

【代码实现】

```
// 陈锋
#include <bits/stdc++.h>
using namespace std;
int A, B, Ans;
typedef long long LL;
// cur: 当前已经生成的数字; last_d: 最后一位数字
inline void dfs(LL cur, LL last_d) {
    if (cur >= A) Ans++;
    for(int d = last_d; d < 10; ++d)
        if (cur * 10 + d < B) dfs(cur * 10 + d, d);
}

int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    while (cin >> A >> B) {
        Ans = 0;
        dfs(0, 1);
        cout << Ans << endl;
    }
    return 0;
}
/*
注意：在递归调用之前先判断是否合法
*/
```

例2-30 【数位统计；DFS】数字的个数 (Amount of Degrees, 牛客NC 50516)

求给定区间 $[X, Y]$ 中满足下列条件的整数个数：这个数恰好等于 K 个互不相等的整数 B 的整数次幂之和。例如， $X=15, Y=20, K=2, B=2$ ，则有且仅有下列3个数满足题意： $17=2^4+2^0$ ， $18=2^4+2^1$ ， $20=2^4+2^2$ 。

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>
typedef long long LL;
using namespace std;
int N, M, K, B, Ans;
// 从右往左构造，每次隔一段距离加一个1
// cl: 1的个数; cur: 当前构造好的数字; pb:B 的幂
void dfs(int cl, int cur, int pb) {
    if (cl == K) {
        if (cur >= N) Ans++;
        return;
    }
    // next_pb 下一个 ai = 1 对应的 B^i
    for (LL next_pb = cl ? pb * B : 1; cur + next_pb <= M; next_pb *= B)
        dfs(cl + 1, cur + next_pb, next_pb);
}
int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    cin >> N >> M >> K >> B;
    dfs(0, 0, 0);
    cout << Ans << endl;
    return 0;
}
/*
注意：本题说明了 DP 和搜索问题的紧密联系
*/

```

本节例题列表

本节讲解的例题及其囊括的知识点，如表2-3所示。

表2-3 搜索算法例题归纳